PROJECT BASED LEARNING | REVIEW 3

Hotel Reservation & Guest Services Management System

SQL Queries, Reports, Working Prototype, Testing and Results

Subject	Phase
Database Management Systems	Review 3 of 3 (Final)

1. Recap and Objective of Review 3

Review 1	Review 2	Review 3 (This Document)
Problem, scope, users, rules	Functional dependencies	SQL queries and reports
ER diagram, initial schema	Normalization to 3NF	Complete application workflow
	DDL, constraints, sample data	Testing: 21 test cases, all passed
		Limitations and future scope

Objective of Review 3

To demonstrate the completed system end to end: a Flask and SQLite application that runs the full hotel workflow, produces SQL-based management reports, enforces the key business rules, and is verified by a documented test plan.

2. System Architecture

The prototype follows a three-layer structure. All business rules run on the server before any SQL is executed, and every query is parameterised to avoid SQL injection.

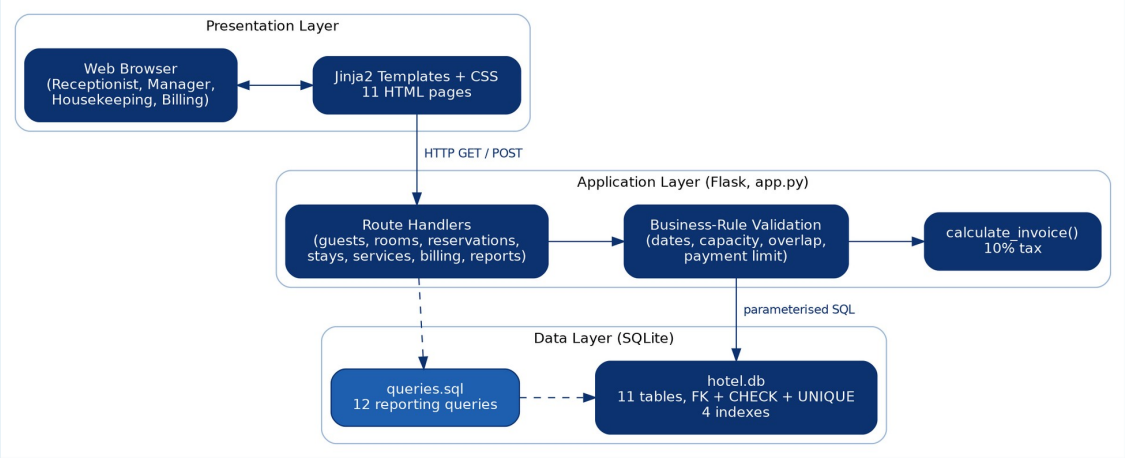


Figure 1. Three-layer architecture of the implemented prototype

Layer	Technology	Responsibility
Presentation	Browser, Jinja2 templates, CSS (11 pages)	Forms, tables, status badges, flash messages
Application	Python 3 and Flask (app.py)	Routing, validation, check-in/out logic, invoice calculation
Data	SQLite (hotel.db), schema.sql, seed.sql, queries.sql	Storage, constraints, reporting queries

3. Implemented Modules

Module	Page	What it does	Main tables
Dashboard	/	Live counts: guests, rooms, available rooms, active reservations, active stays, collected revenue	all
Guests	/guests	Register, search by name, phone or email, edit, safe delete	guest
Rooms	/rooms	Room inventory, add room, change status (Occupied rooms are protected)	room, room_type
Reservations	/reservations	Create, search, filter by status, edit and cancel with full validation	reservation
Stays	/stays	Check-in of eligible reservations and check-out of active stays	stay
Services	/services	Service requests, status updates and one charge per request	service_request, service_charge
Housekeeping	/housekeeping	Task creation, assignment and status; completing a checkout task releases the room	housekeeping_task
Billing	/billing	Invoices, outstanding balance, payments by Cash, Card, UPI or Bank Transfer	invoice, payment
Reports	/reports	SQL-based occupancy, availability, history, revenue and outstanding reports	all
DB Validation	/database-validation	Seven live integrity checks	all

4. Core Workflow Implemented

Guest → Availability → Reservation → Check-in → Stay → Services / Housekeeping → Check-out → Invoice → Payment → Reports

4.1 What happens automatically at check-out

- **Room charge** = number of nights × rate of the reservation's rate plan (minimum one night).
- **Service charge** = sum of all service charges posted to the stay.
- **Tax** = 10 percent of (room charge + service charge); **total** = subtotal + tax.
- The invoice is created, or updated if one already exists, and the stay and reservation become **Completed**.
- The room is set to **Cleaning** and a **Checkout Cleaning** housekeeping task is created. When that task is completed the room returns to **Available**.

4.2 Worked example from the prototype

A guest stays in Room 101 (Standard Flexible, Rs 2,500 per night) for 2 nights and orders a Spa service charged at Rs 1,200. This was run on the application and the invoice below was generated by the check-out button.

Component	Calculation	Amount (Rs)
Room charge	2 nights × 2,500	5,000.00
Service charge	Spa	1,200.00
Tax (10%)	(5,000 + 1,200) × 0.10	620.00
Total	5,000 + 1,200 + 620	**6,820.00**

A first payment of Rs 3,000 moved the invoice to **Partially Paid**; an attempt to pay Rs 6,820 more was **rejected** because it exceeded the outstanding balance; paying the remaining Rs 3,820 changed the status to **Paid**.

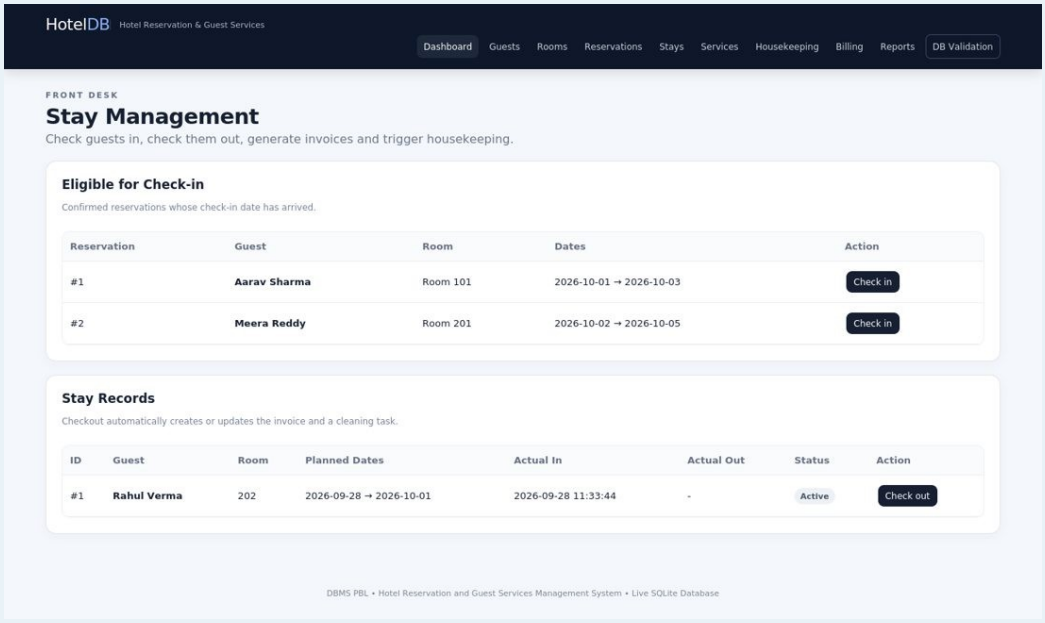


Figure 2. Front Desk: pending check-ins and the active stay list

5. SQL Queries and Reports

database/queries.sql contains 12 reporting queries. The same SQL powers the Reports screen of the application. The results below were produced by running these queries on the seeded database.

#	Report	SQL concepts used	Result on sample data
1	Available rooms	JOIN, WHERE	5 rooms (101, 102, 201, 202, 301)
2	Guest reservation history	JOIN (3 tables), string concatenation, ORDER BY	3 reservations
3	Occupancy / stays per room	LEFT JOIN, GROUP BY, COUNT	Room 202: 1 stay; others 0
4	Cancellation count	COUNT, WHERE	0
5	Housekeeping status	JOIN, ORDER BY	3 tasks
6	Service revenue	SUM, COALESCE	Rs 950.00
7	Invoiced revenue	SUM, COALESCE	Rs 9,295.00
8	Collected revenue	SUM with status filter	Rs 5,000.00
9	Outstanding balances	LEFT JOIN, GROUP BY, HAVING	Invoice 1: Rs 4,295.00 due
10	Revenue by payment method	GROUP BY	UPI: Rs 5,000.00

#	Report	SQL concepts used	Result on sample data
11	Service revenue by type	JOIN, GROUP BY, ORDER BY DESC	Room Service 650; Laundry 300
12	Available room report	JOIN, ORDER BY	Sorted list of available rooms

5.1 Selected queries

Outstanding balance per invoice (uses aggregation and HAVING):

```
SELECT i.invoice_id, i.total_amount,
       COALESCE(SUM(p.amount),0) AS paid,
       i.total_amount - COALESCE(SUM(p.amount),0) AS outstanding
FROM invoice i
LEFT JOIN payment p ON i.invoice_id = p.invoice_id AND p.status='Successful'
GROUP BY i.invoice_id, i.total_amount
HAVING outstanding > 0;
```

Occupancy: stays per room (LEFT JOIN keeps rooms that have never been used):

```
SELECT r.room_number, rt.type_name, r.status, COUNT(s.stay_id) AS total_stays
FROM room r JOIN room_type rt ON r.room_type_id = rt.room_type_id
LEFT JOIN reservation rs ON r.room_id = rs.room_id
LEFT JOIN stay s ON rs.reservation_id = s.reservation_id
GROUP BY r.room_id, r.room_number, rt.type_name, r.status;
```

Overlap detection used when booking (a booking conflicts if it starts before another ends and ends after it starts):

```
SELECT COUNT(*) FROM reservation
WHERE room_id = ? AND status IN ('Pending','Confirmed')
  AND date(check_in_date) < date(:new_check_out)
  AND date(check_out_date) > date(:new_check_in);
```

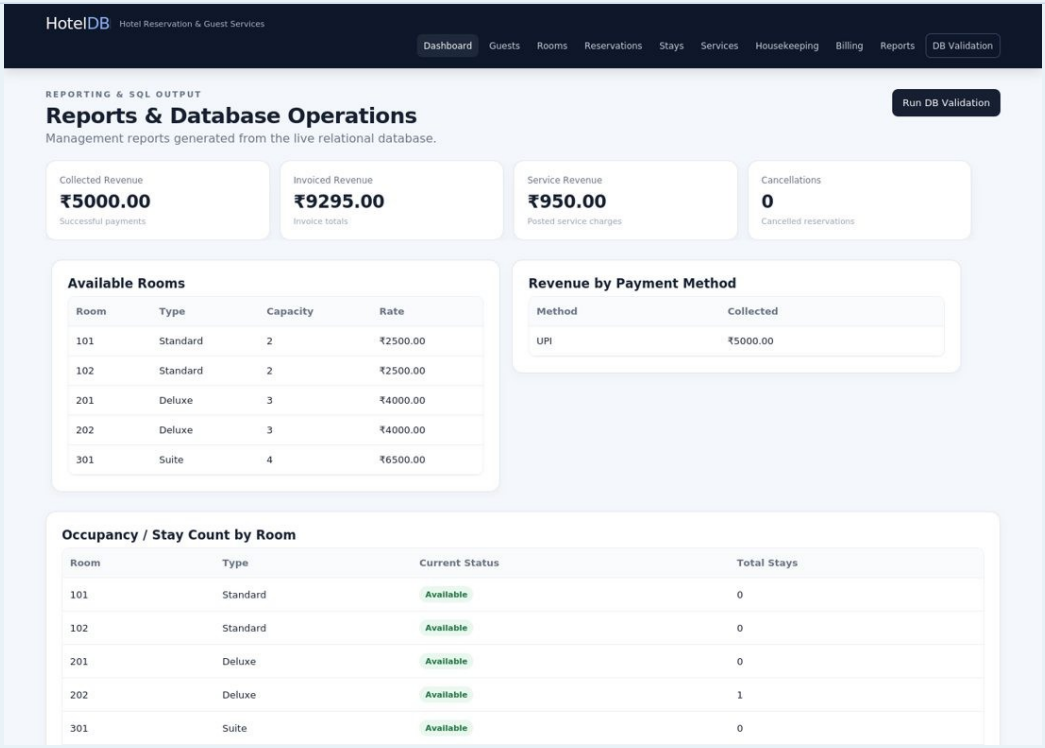


Figure 3. Reports screen: revenue summary, available rooms, payment methods and occupancy

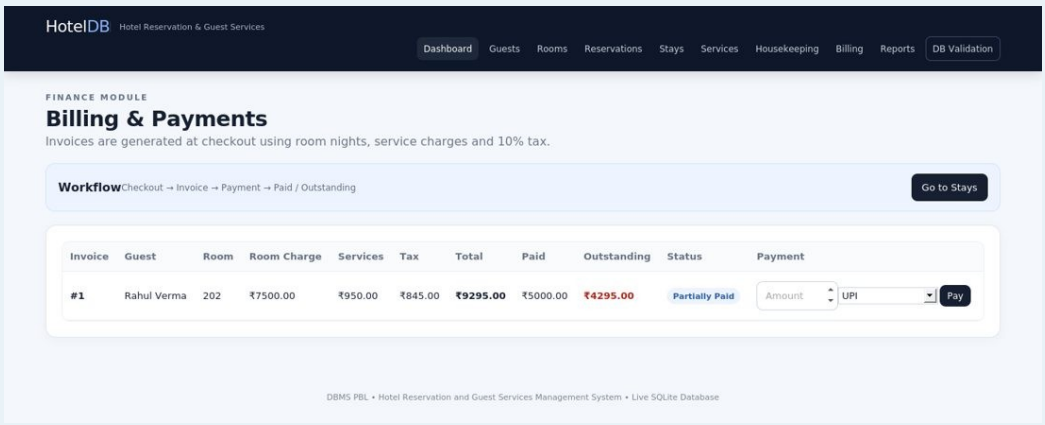


Figure 4. Billing screen: invoice with paid amount, balance and payment form

6. Reliability and Integrity Features

Feature	How it is implemented
Parameterised SQL	Every query passes user input as ? parameters, never by string concatenation (protects against SQL injection)
Foreign keys switched on	PRAGMA foreign_keys = ON on every connection, so SQLite actually enforces references
Rollback on failure	Check-out wraps invoice, stay, reservation, room and task updates together and calls rollback() if any step fails
Safe delete	Guests with reservation history cannot be deleted; the record must be updated instead
State protection	Occupied rooms cannot be manually changed; only active stays can receive service requests
Duplicate protection	Partial unique index allows only one charge per service request

Feature	How it is implemented
Live validation page	Seven checks: foreign keys, required IDs, unique emails, capacity, date ranges, payment limits, reservation overlaps
JSON endpoint	/api/stats returns live counts (guests, rooms, available rooms, active stays, outstanding invoices)

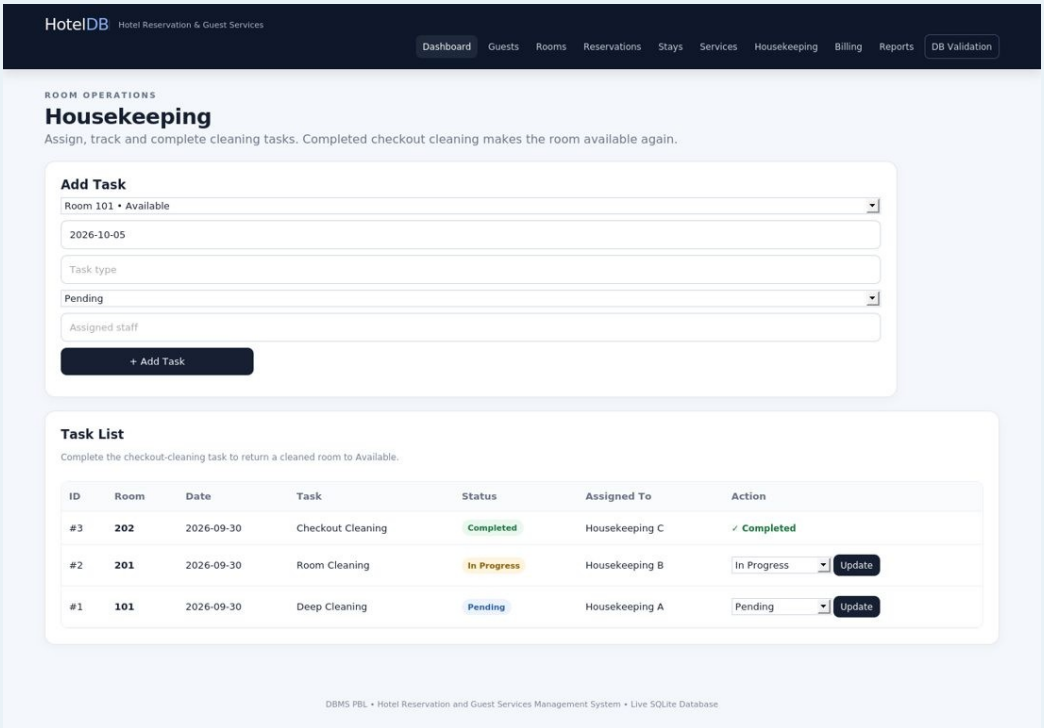


Figure 5. Housekeeping screen: task assignment and status tracking

7. Testing

A test plan of 21 cases (tests/test_cases.md) covers every module. All 21 cases were executed against a copy of the prototype using Flask's test client and direct checks on the SQLite database. **Result: 21 of 21 passed.**

ID	Module	Test	Expected	Actual result	Status
TC01	Guests	Add guest with unique email	Guest inserted	Guest added successfully	PASS
TC02	Guests	Add duplicate email	Rejected by UNIQUE	Duplicate rejected with message	PASS
TC03	Guests	Search by name, phone or email	Matching guests shown	Only the matching guest listed	PASS
TC04	Guests	Edit guest	Updated values stored	Phone number updated in database	PASS
TC05	Rooms	Duplicate room number	Rejected by UNIQUE	Room number must be unique	PASS
TC06	Rooms	Change non-occupied room status	New status stored	Status changed to Maintenance	PASS
TC07	Reservations	Check-out not after check-in	Validation message	Check-out must be after check-in	PASS
TC08	Reservations	Check-in date in the past	Validation message	Check-in date cannot be in the past	PASS
TC09	Reservations	Guests exceed capacity	Validation message	Selected room capacity is 4	PASS

ID	Module	Test	Expected	Actual result	Status
TC10	Reservations	Overlapping reservation	Validation message	Room already has an overlapping reservation	PASS
TC11	Reservations	Edit reservation	Booking updated	Check-out date updated	PASS
TC12	Reservations	Cancel confirmed booking	Status Cancelled	Status changed to Cancelled	PASS
TC13	Stays	Check in eligible reservation	Stay Active, room Occupied	Room 101 became Occupied	PASS
TC14	Stays / Billing	Check out active stay	Invoice made, room Cleaning, task created	Invoice Rs 6,820; room Cleaning	PASS
TC15	Services	Add service request	Request inserted	Service request added	PASS
TC16	Services	Second charge for same request	Rejected	Charge already posted for this request	PASS
TC17	Housekeeping	Complete checkout cleaning	Room becomes Available	Room 101 Available again	PASS
TC18	Billing	Pay within balance	Payment recorded	Invoice became Partially Paid	PASS
TC19	Billing	Pay more than balance	Rejected	Payment exceeds outstanding balance	PASS
TC20	Reports	Open reports	Live SQL values shown	Page loaded with correct totals	PASS
TC21	DB Validation	Run integrity checks	All checks pass	7 of 7 checks passed	PASS

All 11 application pages and the JSON endpoint also returned HTTP 200 during the run.

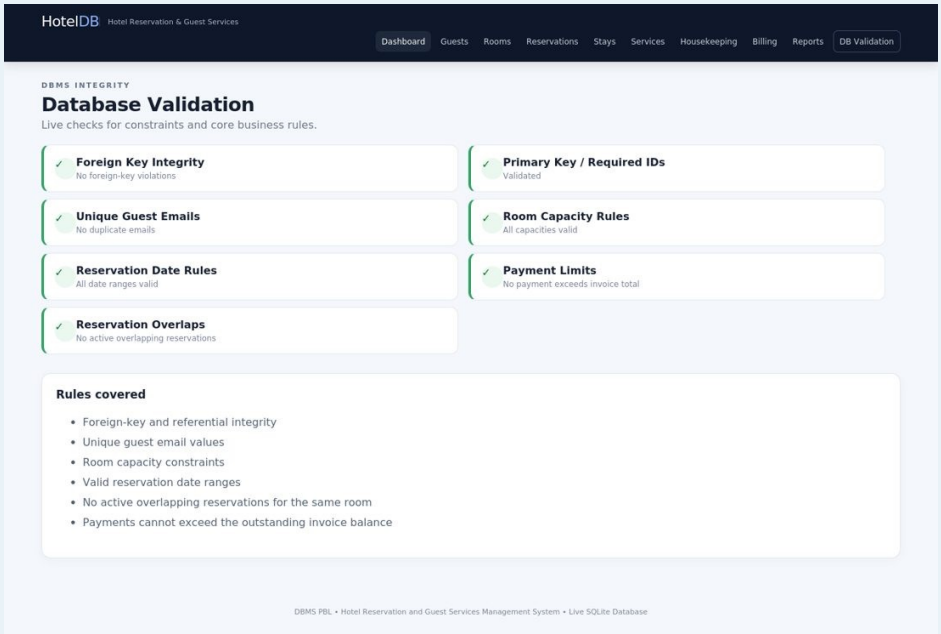


Figure 6. Database Validation: all seven integrity checks passing

8. Challenges, Limitations and Future Scope

8.1 Limitations observed during testing

- **Overlap rule is enforced by the application**, not by a database trigger, so data inserted directly through SQL could bypass it.

- **Connection handling:** a few error paths (for example a duplicate email) return without closing the SQLite connection. During testing this briefly caused a "database is locked" message on the next write. Using Python context managers (with statements) removes this risk.
- **Flat 10 percent tax** and **no login or roles** (receptionist, manager, billing staff and housekeeping are listed in Review 1 but not yet access-controlled).
- **Sample data:** the seeded invoice for the active stay uses Rs 7,500 as room charge; the check-out routine recalculates it from the rate plan, so the final invoice is always correct.

8.2 Future scope

- Role-based login with separate dashboards for the five user types.
- Triggers or stored procedures for overlap and payment rules; migration to MySQL or PostgreSQL for concurrent use.
- Tax configuration table, discounts and seasonal rate plans.
- Online guest booking portal, email or SMS confirmation and payment gateway integration.
- Occupancy percentage and monthly revenue charts on the Reports screen.

9. Conclusion

Across the three reviews the project moved from problem definition and ER design (Review 1), to a normalized 3NF database with constraints and sample data (Review 2), to a fully working application with SQL reports and verified behaviour (Review 3). The final system manages reservations, stays, services, housekeeping, billing and reporting from one centralized database, enforces the six key rules, and passed all 21 planned test cases.

> **Project Summary**
>
> **11** tables | **66** attributes | **12** SQL reports | **10** application modules | **7** live integrity checks | **21/21** test cases passed
>
> Technology: Python Flask, SQLite, Jinja2, HTML/CSS